

NIR2FPGA: Compiling Spiking Neural Networks into RTL

Michail Rontionov

University of Southampton

30th April 2026

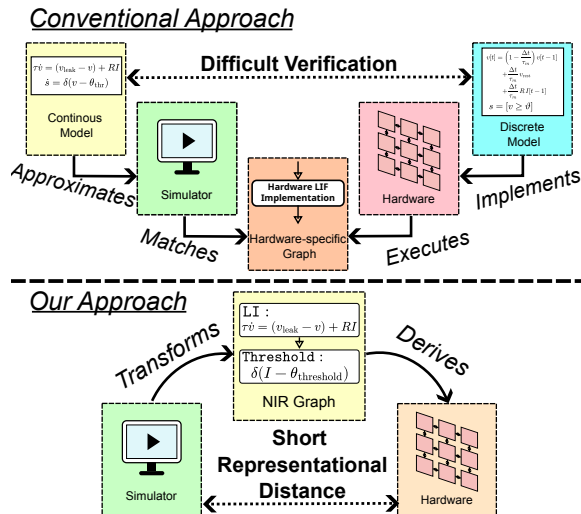


Table of Contents

Hypothesis 1

H_1

In the context of neuromorphic FPGA/ASIC co-design, it is **enough to implement the primitives and connections** between them to **describe the majority of** high-level **SNNs** whilst meeting a given accuracy metric.



NIR2FPGA: Generating Dataflow Accelerators from the Neuromorphic Intermediate Representation

Michail Rontionov^{1†}, Francisco Ayala Le Brun², Nassim Beladel³,
David Thomas¹, Charlotte Frenkel⁴, Jens Egholm Pedersen^{5‡}

¹University of Southampton, Southampton, UK ²Independent Researcher, Netherlands

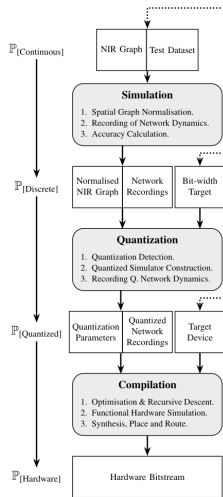
³ETH Zürich, Zürich, Switzerland ⁴Delft University of Technology, Netherlands

⁵Technical University of Denmark

Emails: [†] m.rontionov@soton.ac.uk, [‡] jegpe@dtu.dk

What does it do?

- ▶ Example scenario: “I have an SNN with the *CuBa-LIF* neuron model, implemented in a *NIR-supported simulator*. I would like to evaluate its edge performance in hardware”.
- ▶ Questions arising from using other FPGA works: What source simulators do they assume? Which differential model did they use? Which discretization method did they use? How can I optimise their quantization for my network?
- ▶ Inputs:
 - ▶ NIR Graph (exported from simulator).
 - ▶ Dataset.
 - ▶ Discretization choices (discretization method (e.g. Euler), quantization options, bit width targets)
- ▶ Outputs:
 - ▶ Hardware Bitstream according to inputs.



We make the definitions

- ▶ A **primitive** $p \in \mathbb{P}$ is defined by its inputs, parameters, bounding differential equations and outputs.
- ▶ A **computational model** $\mathcal{M}$ specifies what regime primitive p is evaluated in, for example: *ODE*, *Discrete* equation, *Quantized* equation with discrete model, or in a digital *Circuit*.
- ▶ A **primitive mapping** $\mathbb{P}_{[c]}$ maps a primitive set $\mathbb{P}$ into a computational model $\mathcal{M}$.

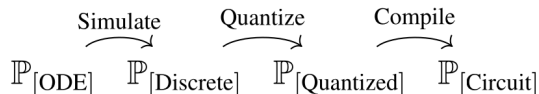
We make the definitions

- ▶ A **primitive** $p \in \mathbb{P}$ is defined by its inputs, parameters, bounding differential equations and outputs.
- ▶ A **computational model** $\mathcal{M}$ specifies what regime primitive p is evaluated in, for example: *ODE*, *Discrete* equation, *Quantized* equation with discrete model, or in a digital *Circuit*.
- ▶ A **primitive mapping** $\mathbb{P}_{[c]}$ maps a primitive set $\mathbb{P}$ into a computational model $\mathcal{M}$.

The paper aims to implement the transformation

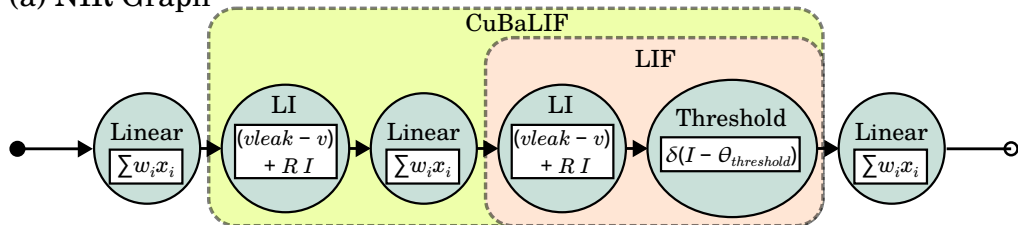
$$\mathbb{P}_{[ODE]} \rightarrow \mathbb{P}_{[Circuit]}$$

We model this as three separate unidirectional transformations



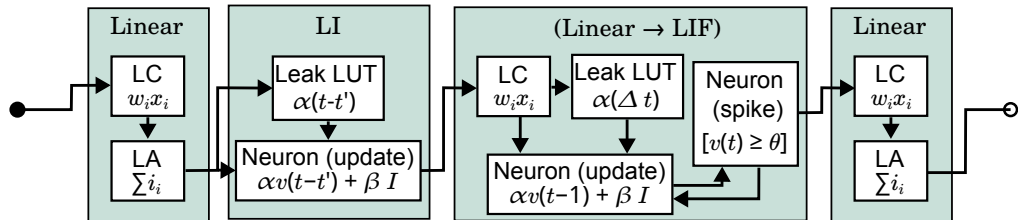
Example Generation

(a) NIR Graph



(a) NIR Graph

(b) Generated Accelerator



(b) Genera

Preliminary Results

Network Representation	W/V Bits	Accuracy	Drop in Acc.	Time/Inf.	LUTs	DSPs	BRAMs
Sinabs Simulator	—	73.28%	0%	—	—	—	—
NIR2FPGA (implicit support)	16/16	63.56%	-9.72%	7211us	2386	24	38
NIR2FPGA (implicit support)	8/8	55.67%	-17.61%	6981us	1935	16	20
SNN-Torch	—	75.44%	0%	—	—	—	—
Spiker+	5/6	72.99%	-2.45%	540us	18268	0	51

- ▶ Not great, but the upside is this heavily depends on FPGA implementation.
- ▶ High discrepancy justifications (and things we plan on addressing):
 - ▶ No formal verification of primitive hardware units.
 - ▶ Lack of thorough equivalence testing (we base on one sample).
 - ▶ Basically, we're not testing as much as we should.
- ▶ MNIST LIF network results are slightly better: -0.94% for 16 bits, -5.08% for 8 bits.

References I